Contents

1 Data as Field Participants: Binding Records, Relationships, and Metrics into Interface Fields **1**

1.1 Abstract . . . 1
1.2 1. Introduction . . . 2
1.2.1 1.1 The problem: relational data, flat rendering . . . 2
1.2.2 1.2 The binding answer . . . 2
1.2.3 1.3 Contributions . . . 3
1.3 2. Background and related work . . . 3
1.4 3. The binding model . . . 4
1.4.1 3.1 Records → bodies . . . 4
1.4.2 3.2 Relationships → graph edges . . . 4
1.4.3 3.3 Record state → metrics and feedback . . . 5
1.4.4 3.4 Feedback → DOM (the reverse half) . . . 6
1.5 4. The lifecycle . . . 6
1.5.1 4.1 Deterministic id-diffed updates . . . 6
1.5.2 4.2 Updates as field transitions . . . 6
1.5.3 4.3 Removal as release / decay . . . 6
1.6 5. Patterns over bound data . . . 7
1.7 6. Shipped demonstrations . . . 7
1.7.1 6.1 Evidence Field — claims, sources, support, contradiction . . . 7
1.7.2 6.2 Search Field — relevance and confidence as a gradient . . . 8
1.7.3 6.3 Review Field — review pressure over a pull request . . . 8
1.7.4 6.4 System Weather — aggregate risk and urgency . . . 8
1.7.5 6.5 What ships, and what does not . . . 8
1.8 7. Evaluation: a design argument and a sketch . . . 8
1.8.1 7.1 What the binding buys (a feasibility argument) . . . 8
1.8.2 7.2 A study sketch (a plan, not a result) . . . 9
1.9 8. Limitations . . . 9
1.10 9. Discussion . . . 10
1.11 10. Conclusion . . . 10
1.12 Appendix A. Reproducibility . . . 10
1.13 Appendix B. Conversion notes (markdown → preprint) . . . 11
1.14 Citation coverage . . . 11

1 Data as Field Participants: Binding Records, Relationships, and Metrics into Interface Fields

> **Status: research draft (preprint, work in progress).** Paper 7 of the Fundamental family — the data-binding validator. Claims verified against the codebase and canonical docs as of 2026-06-07. See the series index and *the caveat canon* therein. This is a preprint draft, not canonical product documentation.

Author: Zach Shallbetter **Series:** Fundamental Research Papers, Paper 7 of 8 **Companion paper (the flagship):** Fundamental: A Field Translation Runtime for Relational DOM Interfaces. See the series index.

1.1 Abstract

Application data — search results, review queues, dashboard tiles, claims and their sources — is inherently *relational* and *stateful*: a result is more relevant than its neighbors, a file blocks a merge, a claim is contradicted by a source. Yet the Document Object Model renders such data as a flat list of boxes, and

the relationships, priority, and uncertainty that motivated the data in the first place live nowhere in the rendered interface; they are re-derived, inconsistently, by bespoke per-view code. This paper presents the **data-binding model** that lets application data *itself* become a field substrate. The central claim is narrow and concrete: when records map to **bodies**, record links map to **graph edges**, and record state maps to **metrics and feedback**, a list, a dashboard, or a result set behaves as a *relational field* — not a flat collection. We describe the shipped mechanism, `bindData()` (`packages/dom/src/bind-data.ts`), as it actually exists: a per-record *mapper* that yields body tokens, metric values, and typed relationships; deterministic **id-diffed** updates (add / update / remove); and **decay on removal**, so a departing record eases out of the field rather than popping. We show that `bindData()` composes with the pattern runtime (Paper 6) — the pattern supplies the *behavior*, the binding supplies the *participants* — through an `annotateBodies: false` path that binds a pattern's metrics while leaving the data's own body tokens intact. We document four shipped demonstrations spanning evidence, search, code review, and operations dashboards, each built data-first from the same mechanism. We are explicit about the limits: a binding is only as meaningful as the mapping the host supplies, confidence and risk must be *supplied* and are never invented by the engine, and no controlled user study has yet been run. The contribution is the model and its shipped realization; the empirical validation is framed as a protocol, not a result.

1.2 1. Introduction

1.2.1 1.1 The problem: relational data, flat rendering

The flagship paper (Paper 1) reframes interface state from *local and binary* to *spatial, relational, and reciprocal*, and its taxonomy of field agents already lists a `DataAgent` alongside particles, elements, relationships, events, users, and layout regions (Paper 1, §3.1). This paper takes that one agent class seriously and asks the concrete question it implies: **how does real application data become a field participant?**

The motivating observation is that the data behind most non-trivial interfaces is already a graph of stateful records, but the rendering throws that structure away. A search result set is a *ranking* — each result carries a relevance and a confidence, some results support a query and some contradict it — yet it renders as an undifferentiated `<ul>`. A code review is a *dependency structure* — some changed files carry more risk than others, some block the merge, comments attach to the files they touch — yet it renders as a flat file list plus a separate comment thread, with the attachment relation expressed only as prose. A dashboard is a *risk surface* — some metrics are hot, some anomalous, the aggregate has a mood — yet it renders as a uniform grid of cards. A set of AI-generated claims is an *evidence structure* — claims lean on sources, sources support or contradict, confidence varies — yet it renders as text with citation superscripts.

In each case the relationships, the priority, and the uncertainty are *real properties of the data* and the central content of the user's task, but the DOM has no place to put them. They get re-expressed, if at all, through ad hoc color, ordering, and one-off JavaScript, re-implemented per view and rarely consistent across two views of the same data.

1.2.2 1.2 The binding answer

Fundamental already provides a runtime in which DOM elements participate in a shared relational field as *bodies*, connect through a typed *relationship graph*, and carry measured *metrics* written back as CSS custom properties (Paper 1, §§3, 5; Paper 5 for the runtime). The data-binding model closes the last gap: it makes the *data* the source of those participants. Concretely, a binding maps

- each **record → a body** (a measured `[data-body]` element), via a host-supplied mapper;
- each record's **links → graph edges** in the `RelationshipRegistry`;
- each record's **state fields → metrics and feedback** (`data-field-<metric>` values that the runtime turns into `--field-*` custom properties).

When those three mappings hold, *the data is the field*: the list's relational structure, the ranking's priority,

and the result set's uncertainty become first-class, measurable, and reciprocal, without a parallel hand-maintained graph and without bespoke per-list rendering code.

1.2.3 1.3 Contributions

This paper contributes, for the single claim *data can be a field substrate*:

1. **A data-binding model** (§3): the precise mapping from records, links, and record-state to bodies, edges, and metrics/feedback, grounded line-by-line in the shipped `bindData()`.
2. **A binding lifecycle** (§4): `bindData()` as a function of a *changing* dataset — deterministic id-diffed add/update/remove, updates realized as field transitions, and removal realized as release/decay rather than a pop — as the code actually implements it.
3. **Composition with patterns** (§5) and **four shipped, cross-domain demonstrations** (§6): evidence, search, review, and system-weather study pages, each built data-first from one mechanism.

Scope discipline, per the family's "one claim per paper" rule: the paradigm is Paper 1; the runtime and host architecture are Paper 5; the pattern schema and catalog are Paper 6; evidence and trust are Paper 3; reading is Paper 2; accessibility is Paper 4; diagnostics are Paper 8. This paper cross-references those and does not re-derive them.

1.3 2. Background and related work

Data binding and reactive views. The lineage of *binding application data to a view that updates when the data changes* runs through Model-View-ViewModel and observable collections in desktop UI frameworks, and through the reactive-rendering model of modern web frameworks (a view as a function of state, reconciled on change). `bindData()` shares the surface goal — a declarative map from data to rendered output, updated incrementally — but differs in *what it binds to*: not a component tree whose state is local and binary, but a relational field whose participants carry continuous metrics and typed edges. [gossman2005; fowler2004presentation; react-docs-declarative]

Data-driven documents. The closest methodological ancestor is the data-join of data-driven document toolkits, which bind an array of records to a selection of DOM/SVG nodes and split each update into enter / update / exit sets keyed by identity. `bindData()`'s id-diff (§4) is recognizably in this tradition — added, kept, and removed ids drive node creation, mutation, and teardown — but the *target* of the join is a field participant (a body with tokens, metrics, and relationships) rather than a positional mark, and the *exit* is a physical decay rather than an immediate removal. [bostock2011; d3-selection-join]

Graph and relational visualization. Node-link and force-directed visualization render relational data by *position*: edges become layout constraints that a solver resolves into coordinates. The Fundamental stance (Paper 1, §2) is that force is an *expressive medium* writing state back into arbitrary semantic elements, not only a layout solver; the binding inherits that stance, so a record's edges raise its neighbors' coherence or entropy rather than only repositioning them. [eades1984; fruchterman1991; bostock2011]

List virtualization versus relational structure. A large body of engineering practice optimizes *long flat lists* — windowing, virtualization, recycling — treating the list as a homogeneous sequence to be rendered cheaply. That work is orthogonal and complementary: it answers *how to render many rows*, while the binding model answers *what relational structure the rows carry*. We note the performance tension this creates in the limitations (§8). [react-window; react-virtualized]

The distinguishing stance, across all of these, matches the family's: the binding does not fabricate the relational signal it renders. Confidence and risk are *supplied* by the host, never inferred from the mere presence of a citation (§3.3); the engine's honesty about provenance is a designed property, not an afterthought.

1.4 3. The binding model

The shipped entry point is one function:

```
function bindData<T>(
  container: HTMLElement,
  records: T[],
  mapper: RecordMapper<T>,
  options?: BindDataOptions<T>,
): DataBinding<T>;
```

(`packages/dom/src/bind-data.ts`.) It returns a handle with `update(records)`, `ids()`, `applied()`, `inspect()`, and `destroy()`. The mapper is the heart of the model: it is the host-authored function `(record, index) => MappedRecord` that performs the three mappings below. Every mapping in this section is exactly what the mapper's return type permits and what the binding does with it.

1.4.1 3.1 Records → bodies

The mapper turns each record into a `MappedRecord`, whose `body` field is a `MappedBody`:

```
interface MappedBody {
  tokens: string[];        // force tokens → data-body
  strength?: number;       // → data-strength
  range?: number;          // → data-range
  spin?: number;           // → data-spin
  angle?: number;          // → data-angle
  feedback?: boolean;      // → data-feedback (opt into reverse write-back)
}
interface MappedRecord {
  id: string;
  body: MappedBody;
  metrics?: Record<string, number>;
  relationships?: MappedRelationship[];
  label?: string;
}
```

For each record, `bindData` creates (or reuses) one element — a `<div>` by default, or the `tag` option's element — sets its `id` to the record id so it is *addressable* as a relationship target, and applies the mapped body: it writes `data-body` from `body.tokens.join(' ')`, and sets `data-strength`, `data-range`, `data-spin`, `data-angle`, and `data-feedback` from the corresponding optional fields (numeric attributes are removed when the value is absent or non-finite). The element becomes an ordinary field body — exactly the `[data-body]` contract the platform runtime already measures and feeds back (Paper 1, §7.1) — but its tokens and parameters now come from a record rather than from hand-written markup. The mapper may also supply a `label` (rendered into a `.bd-label` span) or, via the `content` option, arbitrary domain HTML per record (rendered into a `.bd-content` box); the search, review, and weather demos use `content` to render real result/file/card markup while the body attributes drive the field underneath it.

The body's *meaning* is entirely the mapper's: in the search demo a result maps to `tokens: ['charge', 'link']` with `strength: 0.4 + r.confidence`; in the weather demo a metric card maps to `tokens: ['gravity', 'thermal', 'pressure']` with `strength: 0.5 + c.heat`. The runtime does not know these are "results" or "cards" — it knows only that they are bodies with these tokens and these strengths. This is the load-bearing move: *importance becomes strength and range; the category of behavior becomes the choice of tokens* (the `DataAgent` mapping the canonical model already names — importance → strength, recency → heat, uncertainty → entropy; `docs/canonical/…interaction-and-relationship-model.md` §15).

1.4.2 3.2 Relationships → graph edges

A record's links are the second mapping:

```
interface MappedRelationship { to: string; type: string; strength?: number; }
```

For each mapped relationship, the binding appends a hidden child anchor (`a.bd-rel`, `aria-hidden`) to the record's element, carrying `href="#<to>"`, `data-field-relation="<type>"`, `data-field-target="#<to>"`, and an optional `data-field-strength`. These anchors are precisely the native and `data-field-relation` signals the platform's `RelationshipRegistry` already discovers and normalizes into a typed relationship graph (Paper 1, §5.2; Paper 5). The binding therefore does not maintain a parallel graph of its own — it *emits the markup the existing relationship discovery already understands*, and the runtime folds those edges into the same `RelationshipAgent` graph that native `a[href#id]`, `aria-controls`, and `data-field-relation` produce. On every record render the binding clears the prior `.bd-rel` anchors and re-emits the current set, so the edge set tracks the data.

A subtlety the runtime handles, and the binding inherits, is **unresolved targets**. A declared edge whose target id-ref resolves to no element is *tracked, not dropped*: the relationship scanner returns both a `resolved` set and an `unresolved` set, and the registry keeps the unresolved declarations so inspection can name each missing endpoint (`packages/dom/src/relationships.ts`). This matters for data binding specifically, because the data may declare an edge to a record that has not yet arrived or has just left. The pattern runtime then counts an unresolved edge toward an element's *total* relationships but not its *resolved* set, so a citation pointing at nothing *lowers* resolution and *raises* entropy rather than silently vanishing (`apply-recipe.ts`; §3.3). The Evidence Field demo relies on exactly this: claims declare `supports`/`contradicts` edges to source list items, and the field reads the resolution honestly.

1.4.3 3.3 Record state → metrics and feedback

The third mapping is the mapper's `metrics?: Record<string, number>`. For each entry the binding writes `data-field-<metric>` on the record's element. These attributes are the input lane the pattern runtime reads: in `applyRecipe`'s compute phase, any `data-field-<metric>` present on a body is read as a *supplied* metric value, clamped to `[0, 1]`, and — critically — *supplied values win* over computed ones (`packages/dom/src/metrics.ts`, `computeMetrics`; `apply-recipe.ts`). The runtime then folds the metric into the element's state and the feedback layer flushes it to a `--field-<metric>` custom property, which the page's CSS consumes. So a record's `{ confidence: 0.8 }` becomes `data-field-confidence="0.8"` becomes `--field-confidence: 0.8` becomes, in the demo CSS, a trust gradient on the result's left border. This is the *reverse half* of the reciprocal loop (Paper 1, §8) driven by data: the record's state parameterizes its own rendering.

The mapping from record fields to metrics is the host's design choice, and the canonical `DataAgent` table names the natural correspondences (`…interaction-and-relationship-model.md` §15): importance → strength, recency → heat, **uncertainty → entropy**, confidence → coherence. Two of these deserve emphasis because the engine is deliberately constrained about them:

- **Confidence is host-supplied, never fabricated.** The metric library is explicit that `confidence` is a *SUPPLIED-ONLY* lane: the engine "has no evidence for a claim's truth, so it stays unset unless the host supplies it" (`metrics.ts`). An earlier default — `confidence = relTotal > 0 ? resolvedRatio : 0`, i.e. "any citation fully confident" — was removed precisely because it was "the wrong default for an evidence/trust surface": a citation is not certainty, a source is not proof. Confidence is present only when the data supplies it; relationship resolution is a *separate* signal, not confidence. (This is the substance of the project's #220 confidence and #222 resolution corrections.)
- **Risk is not inferred.** Like `confidence`, `risk` is a *supplied-only* lane — absent unless the host supplies it (`metrics.ts`); the engine never defaults it to `0` ("no risk" is a claim, not a safe blank). A binding that wants risk must supply it.

When a supplied metric is *removed* between frames — the host stopped supplying `data-field-confidence` — the runtime drops the stale state *and* clears the bound CSS variable, so "absent reads as absent, not last-known" (`apply-recipe.ts`, state phase). Data-driven metrics are therefore honestly transient: they appear when the data justifies them and disappear when it does not.

1.4.4 3.4 Feedback → DOM (the reverse half)

The mappings of §§3.1–3.3 are the forward half (data → field). The reverse half (field → data's rendering) is the same `FeedbackRegistry` flush the flagship describes (Paper 1, §8), now closing the loop over records: the computed/supplied metrics become `--field-*` custom properties on each record's element, and the demo CSS turns them into weight, color, border, and opacity. Nothing in this half is new to the binding — it is the platform's existing write-back — but it is what makes the data-bound list *behave* as a field: a contradicted claim does not merely carry a `contradicts` edge in a graph, it *reads* contested because its entropy rose and its coherence fell and the CSS shows it.

1.5 4. The lifecycle

`bindData()` is not a one-shot render; it is a binding over a *changing* dataset. Calling `binding.update(nextRecords)` re-runs the map against the new array, and the difference between this section's behavior and a naive "clear and re-render" is the whole point: updates are deterministic, and departures decay.

1.5.1 4.1 Deterministic id-diffed updates

Each render maps the records, extracts their ids, and computes the difference against the currently bound ids with a pure `diffIds(prev, next)` that returns `{ added, removed, kept }` (`bind-data.ts`). For every mapped record the binding reuses the existing element if its id is already bound (mutating its body, metrics, content, and edges in place) or creates a new element if the id is new; after each record it re-appends the element to the container so **DOM order stays aligned with record order**. Because elements are keyed by record id rather than by position, an update that reorders or edits records does not tear down and rebuild the unchanged ones — it touches exactly the elements whose data changed. Identity, not array index, is the unit of update. (This is the same discipline that makes a data-join correct under reordering; here it also keeps a body's *field state* continuous across updates, since the element — and thus its measured history and metric prior — survives.)

1.5.2 4.2 Updates as field transitions

Because a kept record's element persists across an update, a change to its mapped metrics or body is a *transition of an existing body*, not a replacement. When a claim's source is flipped from `supports` to `contradicts`, `update()` re-emits that claim's edges; the relationship graph's resolved/conflict counts change; the pattern's compute phase recomputes coherence and entropy for that body; and the feedback flush eases the `--field-coherence`/`--field-entropy` variables to their new values, which the CSS transitions. The record's metric change *re-shapes the field* — the body and its neighbors respond — rather than the list snapping to a new static arrangement. The binding re-applies the pattern only when the *set* of participants changes (an add or a remove), so steady metric updates flow through the already-running pattern loop (§5).

1.5.3 4.3 Removal as release / decay

A removed id is not deleted immediately. The binding marks the departing element `data-bd-exiting`, zeroes every `data-field-*` attribute on it so its feedback eases down, removes it from the live id map (so it is excluded from the pattern's re-application), and schedules its actual DOM removal after a `decayMs` delay (default 400 ms). During that window the CSS fades and translates it out, and its metrics have already relaxed to zero, so a removed record *releases* rather than *pops*. This is the "weak-interaction → transformation" register of the Natural Field Translation System made literal for data (Paper 1, §6.5): a record's departure is *decay, release, expiration, handoff*, not an abrupt disappearance. Entering records are given the symmetric treatment — created with `data-bd-entering`, which the next animation frame removes, so CSS can ease them in. The lifecycle thus has a physical grammar at both ends: arrivals settle in, departures decay out, and the steady state in between is a field of bodies whose metrics track their records.

`destroy()` tears the whole binding down: it destroys the applied pattern (clearing the feedback variables it

wrote, so a torn-down binding leaves the DOM plain — `apply-recipe.ts`) and removes every element. The binding leaves no residue.

1.6 5. Patterns over bound data

`bindData()` supplies the *participants*; a **pattern** (Paper 6) supplies the *behavior*. The two compose through one option and one flag, and the division of labor is clean enough to state in a sentence: the pattern frames *which* metrics are tracked and how they feed back; the per-record mapper owns the body tokens, the metric *values*, and the relationships — so the data drives the field, not a mock.

Internally, when the binding's participant set changes it calls

```
applyRecipe(container, pattern, {
  bodies: items,
  annotateBodies: false,   // keep the data's own data-body tokens
  reducedMotion: options.reducedMotion,
});
```

(`bind-data.ts`, `reapply()`.) The `annotateBodies: false` path is the key to the composition. By default `applyRecipe` *overwrites* each body's `data-body` attributes with the pattern's own body tokens (Paper 6); under `annotateBodies: false` it leaves the caller-owned tokens intact and still binds the pattern's metric→variable framework and discovers relationships (`apply-recipe.ts`, `ApplyRecipeOptions`). This is exactly what data binding needs: the *mapper* decided that a search result is `['charge', 'link']` based on the record, and the *pattern* (`trust-gradient`) decides that `confidence` is the tracked metric and `--field-confidence` is its output variable. Neither overwrites the other. The pattern contributes its compiled metric lane, feedback bindings, relationship discovery, and reduced-motion static surface; the data contributes the bodies that lane runs over.

The pattern's *internals* — validation, compilation, the conformance gate that rejects non-real tokens, the metric/diagnostic catalog — are Paper 6's subject and are deferred here. What matters for the binding claim is only the contract: a `FieldPattern` is the API representation of a Field Pattern — a portable field program — and `bindData()` lets real data be the program's input. The same mechanism therefore inherits the pattern runtime's guarantees — reduced-motion equivalence (a real static surface is installed when motion is reduced; Paper 4), live inspection (`applied().inspect()` reports measurements, resolved/unresolved relationships, and live metric lanes), and the lint self-audit — for free over data.

1.7 6. Shipped demonstrations

Four study pages ship under `apps/site/src/pages/docs/studies/`, each built *data-first*: the page renders nothing static for the field region; a `bindData()` call creates the bodies from records, and toggling "Field: off" re-binds *without a pattern*, so the same records still render as a plain list/grid (the honest "before"). All four share a Field on/off toggle, a reduced-motion toggle, and a live `inspect()` readout (records · bodies · edges), driven by `mountBindStudy()` (`apps/site/src/lib/study.ts`). They demonstrate the *mechanism* across domains; they are concept studies, not controlled experiments (§6.5).

1.7.1 6.1 Evidence Field — claims, sources, support, contradiction

`studies/evidence-field.astro`. Claims are records; sources are static list items that the claims' edges resolve against. The mapper maps each claim to `tokens: ['charge', 'link', 'cohesion']`, `strength: 0.6 + confidence * 0.6`, `feedback: true`, `metrics: { confidence }`, and `relationships: refs.map(r => ({ to: r.src, type: r.kind, strength: r.w }))` where `kind` is `supports` or `contradicts`. The `evidence-field` pattern turns the resolved and conflicting edges into coherence and entropy; well-supported claims read coherent, contradicted claims gain entropy and read contested. The page's controls exercise the full lifecycle: *add claim* and *remove claim* drive id-diffed add/decay; *flip a source* flips a claim's first edge

from `supports` to `contradicts` and `update()`s, re-shaping the field; *reduce motion* rebuilds the binding with the pattern's static surface. Confidence here is *supplied by the claim record*, never inferred from the presence of sources — the §3.3 constraint in action, and the concrete reason this demo is the data substrate for Paper 3's trust protocol.

1.7.2 6.2 Search Field — relevance and confidence as a gradient

`studies/search-field.astro`. Results are records mapped to `tokens: ['charge', 'link']`, `strength: 0.4 + confidence`, `spin: polarity === 'contradict' ? -1 : 1`, `metrics: { confidence, recency }`, with real result markup supplied via `content`. The `trust-gradient` pattern turns each result's confidence into a `--field-confidence` gradient on its border and title color, and a contradicting result stands apart (negative spin, a `contradicts` badge). Opening a result toggles `data-active`, which the runtime reads as engagement. A ranked `<ul>` becomes an evidence landscape in which relevance reads as gravity-like weight and source confidence reads as a trust gradient — over real result records, with no per-result rendering logic beyond the mapper.

1.7.3 6.3 Review Field — review pressure over a pull request

`studies/review-field.astro`. This demo binds *two* groups: changed files (`tokens: ['link', 'tether', 'charge']`, `metrics: { heat, tension }`, via the `dependency-tension` pattern) and reviewers (`tokens: ['charge', 'link']`, `metrics: { attention }`, via the `review-constellation` pattern). Files heat by impact and blocked files hold tension; reviewers form a constellation by attention. Review *comments* are static list items carrying `data-field-relation="affects"` / `data-field-target="#<file-id>"` anchors that resolve to the *data-bound* file elements by id — a relationship from hand-authored markup into the bound data, demonstrating that bound records are first-class relationship targets. The pull request, normally a flat file list plus a detached comment thread, becomes a field where review pressure is visible structure.

1.7.4 6.4 System Weather — aggregate risk and urgency

`studies/system-weather.astro`. Dashboard metric cards are records mapped to `tokens: ['gravity', 'thermal', 'pressure']`, `strength: 0.5 + heat`, `metrics: { heat, pressure }`, with card markup via `content` and an `anomaly` flag rendered as a badge. The `attention-weather` pattern turns supplied heat and pressure into `--field-heat`/`--field-pressure` that the CSS renders as weather: hot or overloaded metrics gain weight and warmth, calm regions stay quiet. The aggregate "system pulse" is computed from the cards' heat. This is the alert-fatigue archetype from the flagship (Paper 1, §8.6) realized over a data-bound grid: priority is field-based, so calm stays calm.

1.7.5 6.5 What ships, and what does not

What ships is the **mechanism** (`bindData()`, `packages/dom/src/bind-data.ts`, landed in #210) and these **four demonstrations**, each verifiable in the repository and each built data-first. What does *not* ship is any **controlled user study**: per the caveat canon (item 6), no user-study results exist in this family yet, and none are claimed here. The studies are concept demonstrations of the binding mechanism; the trust study *protocol* itself belongs to Paper 3, which uses the Evidence Field binding as its substrate. The line is the same one the flagship draws: the demos prove the substrate on familiar interfaces *without spectacle*, and the empirical question is left to a study that has not been run.

1.8 7. Evaluation: a design argument and a sketch

1.8.1 7.1 What the binding buys (a feasibility argument)

The evaluation here is, at this stage, a *design and feasibility* argument, in the family's honest register. Three properties follow from the model and are demonstrated by §6:

1. **Relational structure, priority, and uncertainty become first-class over real data.** The four demos express support/contradiction edges, relevance/heat priority, and confidence/entropy uncertainty — directly from records, through one mechanism — where the conventional rendering expresses none of them. The relationships live in the runtime's typed graph, not in per-view code.
2. **No bespoke per-list code.** Each demo's field behavior is a *mapper plus a pattern*. The mapper is a single pure function from record to `MappedRecord`; the pattern is a named, conformance-gated program (Paper 6). There is no hand-wired animation or per-row state machine; "no hand-wired behavior — the page comes from data" is the literal design note on each study.
3. **Cross-domain reuse.** One mechanism spans evidence, search, code review, and operations dashboards. The differences between the demos are entirely in their mappers and chosen patterns; the binding, the lifecycle, the relationship discovery, and the feedback flush are identical across all four. This is the strongest available evidence that the binding is a *general* substrate and not four bespoke effects wearing one name.

1.8.2 7.2 A study sketch (a plan, not a result)

A controlled evaluation tied to a project target would ask whether data-bound relational structure *helps a real task*. One concrete sketch, deferring the trust-specific protocol to Paper 3: a within-subjects triage or search task in which participants resolve a query against a result set or a review queue, comparing a *plain* rendering (the demos' field-off mode, which is a genuine flat list) against the *data-bound field* rendering of the identical records. Candidate measures: time-to-correct selection, error rate on identifying the most-relevant or most-contradicted item, and recall of *which* evidence supports a decision (the canonical "source binding improves recall" hypothesis from the Evidence demo's own claim list). The field-off / field-on toggle and the deterministic, data-first construction make such an A/B both *possible* and *fair*: the two conditions are the same records and the same mapper, with the pattern attached or not. We state this strictly as a protocol; **no results are reported**, consistent with the caveat canon.

1.9 8. Limitations

1. **A binding is only as good as its mapping (garbage in, garbage out).** The mapper is host-authored and unconstrained in its *semantics*: nothing stops a host from mapping an irrelevant field to `strength` or inventing a relationship the data does not support. The binding makes the mapping *executable and inspectable*, not *correct*. The field faithfully renders whatever relational claim the mapper makes; the responsibility for that claim's validity is the host's.
2. **Confidence and risk must be supplied, not invented.** As §3.3 details, the engine refuses to fabricate `confidence` (it is supplied-only — the #220 correction) and does not invent `risk` (it is supplied-only, never defaulted to `0` — the #226 correction, mirroring #220). This is a deliberate honesty constraint, but it is also a *limitation on the binding's reach*: a data source that lacks a confidence or risk signal cannot have one synthesized by the field, and a host that wants those metrics must bring its own trust/risk model. The binding will not paper over missing evidence.
3. **Large datasets and performance budgets.** Each record becomes a measured DOM body, and the relationship and metric machinery runs per body per frame. The model is built for *relationally rich, moderately sized* sets — a result page, a review's files, a dashboard's tiles — not for tens of thousands of rows. It does not currently integrate with list virtualization (§2), and the interaction between windowing/recycling and a body's continuous field state is unresolved; a virtual row that is recycled loses the measured history that §4.1 preserves. Performance budgeting for large bound sets is future work.
4. **No controlled study yet.** Per the caveat canon (item 6), the demonstrations are concept studies; the §7.2 protocol has not been run, and this paper reports no measured user outcome.

1.10 9. Discussion

The reason "data as field participants" matters is that it **closes the loop from application state to relational interface behavior without a parallel, hand-maintained graph**. In the conventional arrangement, the relational structure of the data exists twice: once in the data model, and once (partially, inconsistently) in whatever ad hoc rendering the view team rebuilt to gesture at it — and the two drift. The binding collapses that duplication: the data model's records, links, and state are mapped *once*, by a pure function, into the runtime's existing body / relationship / metric machinery, and the rendering is then a *consequence* of the field rather than a second source of truth. When the data changes, `update()` re-runs the map and the field transitions; nothing is re-derived by hand.

This also clarifies what the Fundamental paradigm is *for*. The flagship's threat-to-framing note (Paper 1, §10) is that the model pays off on content with latent relational structure and risks "spectacle over meaning" on flat content. Data binding is the sharpest test of that thesis, because it applies the field to data whose relational structure is *already real and already the user's task*. The four demos are deliberately chosen to be archetypes — evidence, search, review, operations — where the relations, the priority, and the uncertainty are the content, not decoration. On such data the field is not an effect layered over a list; it is the list's own structure made visible and reciprocal. The honest counter-position remains the limitation of §8.1: the binding renders the mapping it is given, so the discipline shifts from *can we show this relation* to *is this relation true* — which is exactly where it should sit.

1.11 10. Conclusion

Application data is relational and stateful, but the DOM renders it flat, and the relationships, priority, and uncertainty that are the substance of the user's task end up living nowhere in the interface. This paper presented the data-binding model that lets the data itself be a field substrate: records map to bodies through a host-supplied mapper, record links map to typed graph edges the existing relationship registry discovers, and record state maps to supplied metrics that the runtime feeds back as `--field-*` properties. The model is realized by the shipped `bindData()` — deterministic id-diffed updates, updates as field transitions, removal as decay — and composes with the pattern runtime through `annotateBodies: false`, so a pattern supplies the behavior while the data supplies the participants. Four shipped demonstrations span evidence, search, review, and operations from one mechanism, with no bespoke per-list code. We were explicit that a binding is only as meaningful as its mapping, that confidence and risk are supplied and never invented, that large-set performance is unresolved, and that no controlled study has yet been run. The contribution is the model and its shipped realization; the empirical validation is a protocol, deferred to Paper 3 for trust and sketched here for triage and search. With this, the family's `DataAgent` is no longer a slot in a taxonomy but a working substrate: a list, a dashboard, or a result set that behaves as a relational field, not a flat collection.

1.12 Appendix A. Reproducibility

Every mechanism claim in this paper is checkable against the repository:

- **The binding mechanism:** `packages/dom/src/bind-data.ts` — the `bindData()` function, the `MappedBody` / `MappedRelationship` / `MappedRecord` / `RecordMapper` types, the pure `diffIds()`, `applyMapped()` (body tokens, metric attributes, relationship anchors), and the add/update/decay-on-remove render loop.
- **Patterns over bound data:** `packages/dom/src/apply-recipe.ts` — the `annotateBodies: false` path (keep caller-owned tokens while binding the pattern's metrics), supplied-metric handling in the compute/state phases, the absent-metric clear, the reduced-motion static surface, and the resolved/unresolved relationship inspection.
- **Metric provenance:** `packages/dom/src/metrics.ts` — `METRIC_KINDS`, the SUPPLIED-ONLY `confidence` lane, the supplied-only `risk` lane (never defaulted to `0`), and the "supplied values win" rule.

- **Relationship discovery and unresolved targets:** `packages/dom/src/relationships.ts` — `scanRelationships()` (resolved vs unresolved), `UnresolvedRelationship`, and the `RelationshipRegistry`'s tracking of declared-but-unresolved edges.
- **The four data-bound study pages:** `apps/site/src/pages/docs/studies/evidence-field.astro`, `search-field.astro`, `review-field.astro`, `system-weather.astro`, plus the shared `apps/site/src/lib/study.ts` (`mountBindStudy`).
- **Package export:** `packages/dom/src/index.ts` re-exports `bind-data.ts`.
- **Canonical corroboration:** `docs/canonical/interaction-and-relationship-model.md` §15 (`DataAgent` property→field mapping), §21 (Search), §26 (AI use cases).

The mechanism landed in #210; the data-bound study pages in #213–#214; the confidence-provenance constraint in #220; the real relationship resolution and unresolved-target tracking in #222 (the supplied-only `risk` treatment landed in #226).

1.13 Appendix B. Conversion notes (markdown → preprint)

Notation is kept LaTeX-compatible; the fenced TypeScript blocks are the binding's real interface and should be reproduced verbatim. Figures referenced in prose but not yet drawn — the records → bodies → edges → metrics mapping (§3), the id-diff lifecycle with enter/update/decay (§4), the recipe-over-data composition (§5), and a before/after of one demo (§6) — are produced at conversion time. External citation keys in `references.md` must be resolved and verified before submission; none are fabricated here.

1.14 Citation coverage

- MVVM, observable collections, and the reactive view-as-function-of-state model (data binding lineage; §2): [gossman2005; fowler2004presentation; react-docs-declarative].
- Data-driven documents and the data-join enter/update/exit pattern keyed by identity (§2, §4.1): [bostock2011; d3-selection-join].
- Force-directed and node-link relational visualization; force as layout solver vs expressive medium (§2): [eades1984; fruchterman1991; bostock2011].
- List virtualization / windowing / recycling, and its tension with per-element continuous state (§2, §8.3): [react-window; react-virtualized].
- HCI literature on triage, search relevance judgment, and evidence recall is intentionally not over-cited here; the §7.2 study sketch is framed as a proposed protocol, not a claimed result.